Introduction to Functions

Separation of Concerns

Department of Computer Science
University of Pretoria

19 March 2024

- Structure of a C++ program
- Data types
  - Ordinal - Integer, Boolean and Characters
  - Variable length - Floating point and String
- Operators - Mathematical, Relational and Boolean

```
#include <iostream>

using namespace std;

int main() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
        return 0;
}
```

What do we know about the main function?

```
int main() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
        return 0;
}
```

What happens if we want to test multiple combinations of `x` and `y`?

```
#include <iostream>

using namespace std;

int main() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
        ...
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
        return 0;
}
```

We will create 3 variations of the `isLessThan` function.

1. with no parameters and no return type
2. with two parameters (`x` and `y`) and no return type
3. with two parameters and a Boolean return type

Which variation is not being considered?

Can you write it?

No parameters and no return type

```
void isLessThan() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
}
```

Our `main` function reduces to:

```
int main() {
        isLessThan();
        return 0;
}
```

Two parameters and no return type

```
void isLessThan(int x, int y) {
        cout << x << " < " << y << " = " << (x < y) << endl;
}
```

Our `main` function becomes:

```
int main() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        isLessThan(x,y);
        return 0;
}
```

Two parameters and return type

```
bool isLessThan(int x, int y) {
        return (x < y);
}
```

Our `main` function becomes:

```
int main() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << isLessThan(x,y)
             << endl;
        return 0;
}
```

What are we trying to achieve with functions?

- Limit the repetition of code that could result in errors due to not changing all locations were code is duplicated
- Encapsulate identifiable subtasks together
- Let the functions do the heavy lifting, but remain as generic as possible

In many cases, the decision of what to include in a function is problem specific. The general rule is, *a function must represent one unit of work*.

Each variation in our example has its own advantages and disadvantages.

1. No parameters and no return type - simplifies the `main`
2. Two parameters and no return type
3. Two parameters and a Boolean return type - least useful for this example, why?

Function definition:

```
<ReturnType> <functionName>([ParameterList]) {
    <FunctionBody>     // Implementation of function's operations
    [ReturnStatement] // Included if the returnType is not void
}
```

Function call:

```
    <functionName> ([ActualParameterList]);
```

or

```
    <type> <variableName> = <functionName>([ActualParameterList]);
```

So where should I define the function?

- All functions must be defined before they can be used.
- There are three options:
  - Before the main function in the same file as the main function
  - In the same file as the main function with a forward declaration
  - In a file separate from the main function

```
#include <iostream>

using namespace std;

void isLessThan() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
}

int main() {
        isLessThan();
        isLessThan();
}
```

```
#include <iostream>

using namespace std;

void isLessThan();

int main() {
        isLessThan();
        isLessThan();
}

void isLessThan() {
        int x, y;
        cout << "Please enter values for x and y: " << endl;
        cout << " x = ";
        cin >> x;
        cout << " y = ";
        cin >> y;
        cout << x << " < " << y << " = " << (x < y) << endl;
}
```

Compiling, linking and executing the examples in Linux (where *N* is 1, 2 or 3):

- Compile

  ```
  g++ -c ExampleN.cpp
  ```

- Link

  ```
  g++ ExampleN.o -o ExampleN
  ```

- Execute

  ```
  ./ExampleN
  ```

What will we do after recess

- Creating the `.h` and corresponding `.cpp` files and the necessary compiler directives
- Compiling and linking with a makefile
- Documenting functions
- Conversion of input and return types